Stage 2 — RAG Implementation

Sprint: Sprint 2
Status: In Progress

Goal

Build the first working Retrieval-Augmented Generation (RAG) pipeline using local models and locally stored knowledge.

The target workflow is:

User Question → Embedding Model → Vector Database Search → Relevant Note Chunks → Prompt Construction → Local LLM → Grounded Response

---

RAG Dependencies

The following core packages were installed:

- LlamaIndex
- ChromaDB
- LlamaIndex Ollama embedding connector
- LlamaIndex Ollama LLM connector

LlamaIndex is being used to connect the Python application with the local embedding and language models.

ChromaDB is being used as the persistent local vector database.

The Ollama connectors allow LlamaIndex to communicate with models served through an Ollama-compatible API.

Embedding Model Integration

EmbeddingGemma was connected to Python through LlamaIndex using the OllamaEmbedding class.

The tested workflow is:

Python → LlamaIndex OllamaEmbedding → Ollama → EmbeddingGemma → Embedding Vector

EmbeddingGemma produces 768-dimensional vectors.

Two important embedding methods were examined:

get_text_embedding()
Used to generate embeddings for stored notes and document chunks.

get_query_embedding()
Used to generate embeddings for user search queries.

Both produce vectors in the same embedding space, but they represent different roles during retrieval.

ChromaDB

A persistent local ChromaDB database was created under:

data/chroma/

Using a PersistentClient means the database remains stored locally after Python, VS Code, or the computer is restarted.

The Chroma database directory was added to .gitignore so that local knowledge and vector data are not committed to GitHub.

A collection named notes was created.

Each Chroma record stores:

- Unique ID
- Original document or chunk text
- Embedding vector
- Metadata

Vector Storage and Retrieval

The vector database logic is stored in:

backend/app/rag/vector_db.py

This module currently provides two main operations:

store_chunk()

The function receives:

- chunk_id
- chunk_text
- metadata

The workflow is:

Chunk Text → EmbeddingGemma → Embedding Vector → ChromaDB

Each chunk therefore becomes an independently searchable vector record.

search_notes()

The function receives a user query.

The workflow is:

Query → Query Embedding → ChromaDB Similarity Search → Matching Chunks

ChromaDB returns information including:

- IDs
- Documents
- Metadata
- Distances

Lower distance values represent closer vector matches.

Semantic retrieval was successfully tested using notes on different topics. A question about how neural networks calculate gradients correctly ranked the stored backpropagation note above unrelated notes even though the wording was different.

This confirmed that retrieval is based on semantic similarity rather than exact keyword matching.

Chunking

Text chunking was implemented using LlamaIndex's SentenceSplitter.

The chunking logic is stored in:

backend/app/ingestion/chunker.py

The current configuration uses:

- Chunk size: 512 tokens
- Chunk overlap: 50 tokens

The workflow is:

Long Note → SentenceSplitter → Multiple Smaller Chunks

Chunk overlap preserves some context between neighboring chunks so that information near a chunk boundary is less likely to be lost.

The chunker was tested with longer text and successfully produced multiple chunks.

Chunk IDs

Each original note has a note_id.

However, each chunk must have its own unique ChromaDB record ID because every chunk is stored as a separate vector record.

Chunk IDs are generated using the format:

{note_id}_chunk_{chunk_index}

Example:

Original note ID:

data220_week1

Generated chunk IDs:

- data220_week1_chunk_0
- data220_week1_chunk_1
- data220_week1_chunk_2

The Chroma ID identifies the individual chunk, while the original note_id is also stored inside the chunk metadata.

This allows every chunk to remain associated with its original note.

Metadata

Each chunk currently stores the following metadata:

- source
- note_id
- chunk_index
- date_added

Example:

Source: manual_test
Note ID: data220_week1
Chunk Index: 2
Date Added: 2026-08-26

The chunk index preserves the original position of the chunk within the note.

This may later allow the retrieval system to find neighboring chunks or reconstruct sections of the original document in the correct order.

Ingestion Pipeline

The ingestion logic is stored in:

backend/app/ingestion/ingestion.py

The ingestion layer connects the chunker with the vector database.

The workflow is:

Full Note → ingest_note() → chunk_text() → Loop Through Chunks → Generate Chunk IDs → Create Metadata → store_chunk() → EmbeddingGemma → ChromaDB

This means a long note can now be passed into the system and automatically transformed into multiple searchable vector records.

Retrieval Validation

Retrieval was tested using notes that had passed through the complete ingestion pipeline.

The validated workflow is:

Long Note → Chunking → Chunk IDs and Metadata → EmbeddingGemma → ChromaDB Storage → User Query → Query Embedding → Similarity Search → Relevant Chunk

Different questions targeting different sections of the long note successfully retrieved semantically appropriate chunks.

This confirms that the complete retrieval side of the RAG system is working.

Project Structure Update

As the project grew, ingestion and retrieval responsibilities were separated.

The relevant structure is now:

backend/app/ingestion/

- chunker.py
- ingestion.py
- ingestion tests

backend/app/rag/

- vector_db.py
- retrieval/vector database tests

backend/app/models/

- model interface logic

backend/app/agent/

- future orchestration logic

backend/app/memory/

- future conversation memory

backend/app/api/

- future backend API

The responsibilities are:

ingestion/
Prepares knowledge for storage.

rag/
Handles vector storage and retrieval.

models/
Handles communication with local AI models.

agent/
Will coordinate retrieval, prompt construction, and generation.

memory/
Will later manage persistent conversation memory.

api/
Will later expose the agent through backend endpoints.

Project imports were also standardized around the app package so modules can use consistent absolute imports such as:

app.ingestion...

app.rag...

app.models...

Local LLM Integration

The LlamaIndex Ollama LLM connector was installed.

A reusable model interface was created under:

backend/app/models/llm.py

The LlamaIndex Ollama class is used to communicate with a locally running language model.

A reusable response-generation function was created.

The basic workflow is:

Python → LlamaIndex Ollama Connector → Ollama → Qwen → Generated Response

This connection was successfully tested.

Official Ollama Performance Investigation

The original language model was:

qwen3.5:4b

It worked successfully through official Ollama, but inference was very slow.

ollama ps showed:

PROCESSOR: 100% CPU

Ollama debug logs also showed:

runner.vram = 0 B

This confirmed that the Intel Arc GPU was not being used.

The model initially loaded with a very large context window of approximately 262,144 tokens.

This caused loaded memory usage to increase to roughly 12 GB.

The context window was reduced to approximately 4,096–8,192 tokens using the context configuration.

After this change, loaded model memory dropped to roughly 3–3.4 GB.

However, Qwen still ran entirely on the CPU.

Official Ollama's Vulkan and integrated-GPU options were also tested, including:

OLLAMA_VULKAN=1

OLLAMA_IGPU_ENABLE=1

The Intel Arc GPU was still not selected.

Development Hardware

The system being used for development contains:

CPU:
Intel Core Ultra 9 285H

GPU:
Intel Arc 140T integrated GPU

Official Ollama could successfully run Qwen on the CPU but did not successfully offload the model to the Intel Arc GPU.

IPEX-LLM Investigation

IPEX-LLM was tested as an alternative runtime because it is optimized specifically for Intel hardware.

A portable Ollama-compatible IPEX-LLM runtime was installed.

The newer model:

qwen3.5:4b

could not be loaded successfully by the IPEX runtime because of model/runtime compatibility.

A different model was then tested:

qwen3:4b

This model successfully loaded and generated responses.

Intel graphics monitoring confirmed that the IPEX runtime was using the Intel Arc 140T GPU.

The IPEX version of ollama ps still reported 100% CPU, but external Intel GPU monitoring confirmed that GPU acceleration was active.

Python + IPEX-LLM

The existing Python LLM interface was then tested against the IPEX Ollama-compatible server.

The working generation path became:

Python → LlamaIndex → IPEX Ollama-Compatible API → Qwen3 4B → Intel Arc 140T GPU → Generated Response

This means the application can continue using the Ollama-compatible API while the underlying LLM inference is accelerated through IPEX-LLM.

EmbeddingGemma Compatibility Issue

EmbeddingGemma was also tested through the IPEX server.

The Python request successfully reached the IPEX endpoint:

POST /api/embed

However, the IPEX model runner terminated while attempting to load EmbeddingGemma.

The server returned:

llama runner process has terminated: exit status 2

and HTTP status:

500 Internal Server Error

This confirms that Python and LlamaIndex successfully contacted the IPEX server, but the current IPEX runtime could not successfully run EmbeddingGemma.

The current runtime situation is therefore:

Official Ollama

- EmbeddingGemma: Works
- Qwen3.5 4B: Works, but CPU-only

IPEX-LLM Ollama

- Qwen3 4B: Works with Intel GPU acceleration
- EmbeddingGemma: Currently fails to load

Current Result

The retrieval side of the system is working:

Long Note → Chunking → EmbeddingGemma → ChromaDB → Semantic Retrieval

The generation side is also working independently:

Prompt → LlamaIndex → IPEX-LLM → Qwen3 4B → Intel Arc GPU → Response

The remaining challenge is allowing both model paths to operate together.

The planned complete RAG workflow is:

User Question → EmbeddingGemma → ChromaDB → Relevant Chunks → Prompt Builder → Qwen3 4B → Grounded Response

Before completing that integration, the runtime configuration needs to support EmbeddingGemma through official Ollama while Qwen3 4B runs through the GPU-accelerated IPEX-LLM server.

Dual Ollama Server Configuration

To allow the embedding model and language model to run through different runtimes, two Ollama-compatible servers were configured simultaneously.

The final runtime architecture is:

Port 11434

IPEX-LLM Ollama-compatible server

Model:

qwen3:4b

Purpose:

Local language model generation with Intel Arc GPU acceleration.

Port 11435

Official Ollama server

Model:

embeddinggemma:latest

Purpose:

Generate document and query embeddings.

The resulting architecture is:

User Question → EmbeddingGemma through Official Ollama on Port 11435 → ChromaDB Retrieval → Retrieved Text → Qwen3 4B through IPEX-LLM on Port 11434 → Generated Response

The two servers were verified independently using their Ollama API endpoints.

The listening processes were also checked to confirm that each port belonged to a different Ollama runtime.

The verified configuration was:

11434 → IPEX-LLM

11435 → Official Ollama

This allowed both runtimes to remain active at the same time.

Explicit Model Routing

The Python application was updated so that each model explicitly communicates with its intended runtime.

The EmbeddingGemma configuration in:

backend/app/rag/vector_db.py

was configured to use:

http://127.0.0.1:11435

The embedding workflow is therefore:

Python → LlamaIndex OllamaEmbedding → Official Ollama:11435 → EmbeddingGemma

The Qwen configuration in:

backend/app/models/llm.py

was configured to use:

http://127.0.0.1:11434

The generation workflow is therefore:

Python → LlamaIndex Ollama LLM Connector → IPEX-LLM:11434 → Qwen3 4B → Intel Arc GPU

This solved the previous compatibility problem by allowing each model to run through the runtime that supports it.

The two model paths no longer need to use the same Ollama server.

LLM Timeout Adjustment

During LLM testing, some requests exceeded the original request timeout.

The original timeout was:

120 seconds

This resulted in:

httpx.ReadTimeout

The timeout was increased to:

300 seconds

This gives the local model additional time to complete generation when necessary.

After the adjustment, Qwen successfully returned responses through the Python LLM interface.

The timeout controls how long the Python client waits for a response.

It does not control how long the generated response is.

Qwen Thinking Output

Qwen3 produces a reasoning section before its final response using:

<think>

and:

</think>

This resulted in responses containing both internal reasoning-style text and the final answer.

An attempt was made to disable this behavior through the LlamaIndex Ollama configuration using:

thinking=False

However, the IPEX-LLM Ollama-compatible runtime did not honor the setting.

Qwen's:

/no_think

instruction was also tested.

The current IPEX runtime treated /no_think as ordinary prompt text rather than disabling the thinking mode.

This indicates that the current older IPEX-LLM Ollama-compatible runtime does not fully support the newer Qwen/Ollama thinking controls.

The LLM itself still generates valid final responses.

Cleaning or suppressing the visible <think> output can therefore be handled later at the application layer if necessary.

Prompt Builder

A prompt construction layer was added under:

backend/app/agent/prompt_builder.py

The purpose of the Prompt Builder is to combine the user's question with the text retrieved from ChromaDB.

The Prompt Builder does not work with embedding vectors.

Embedding vectors are only used to perform similarity search.

After ChromaDB finds relevant records, the stored document text is extracted and passed into the Prompt Builder.

The function receives:

- User question
- Retrieved text chunks

The retrieved chunks are represented as:

list[str]

The chunks are joined together to create a context section.

The basic workflow is:

Retrieved Chunks → Join Text → Add Instructions → Add User Question → Final Prompt

The resulting prompt has a structure similar to:

Instructions

Context:
Retrieved Chunk 1

Retrieved Chunk 2

Retrieved Chunk 3

Question:
User Question

Answer:

The Prompt Builder therefore performs structured string assembly rather than semantic retrieval or generation.

It prepares the information in a format that can be sent to the language model.

Prompt Builder Testing

The Prompt Builder was first tested independently from the rest of the RAG system.

Fake retrieved chunks and a sample question were passed directly into:

build_prompt()

The resulting prompt was printed and inspected.

The test confirmed that:

- The user question was included correctly.
- All retrieved chunks were included.
- Retrieved chunks were grouped into the context section.
- The function successfully returned a complete prompt string.

Testing the Prompt Builder independently helped isolate prompt construction from retrieval and LLM generation.

RAG Orchestration

The individual RAG components were then connected through the agent layer.

The orchestration logic is stored in:

backend/app/agent/ask.py

A higher-level function was created:

ask(question)

The function currently requires only the user's question.

Its workflow is:

User Question

→ search_notes(question)

→ ChromaDB Query

→ Extract Retrieved Documents

→ build_prompt(question, retrieved_chunks)

→ Construct Final Prompt

→ generate_response(prompt)

→ Qwen3 4B

→ Return Final Answer

The responsibilities remain separated between modules.

search_notes()

Handles semantic retrieval.

build_prompt()

Handles prompt construction.

generate_response()

Handles communication with the local LLM.

ask()

Coordinates the complete process.

This keeps each component focused on a single responsibility while allowing the agent layer to connect them.

Complete End-to-End RAG Workflow

With the addition of the Prompt Builder and ask() orchestration function, the complete first RAG pipeline became operational.

The full workflow is now:

User Question

→ ask()

→ search_notes()

→ EmbeddingGemma

→ Query Embedding

→ ChromaDB Similarity Search

→ Retrieved Text Chunks

→ build_prompt()

→ Context + Question

→ generate_response()

→ Qwen3 4B

→ Generated Answer

The model therefore receives the retrieved text rather than the embedding vectors themselves.

The embeddings are used only for locating relevant knowledge.

The actual retrieved text is inserted into the LLM prompt.

End-to-End RAG Validation

The completed ask() pipeline was tested using questions against the stored knowledge.

One test asked about LLM functionality.

The nearest retrieved chunks contained information about general AI models but did not contain sufficient information specifically explaining LLM functionality.

Instead of filling the missing information using general model knowledge, Qwen responded that the provided context did not contain information about LLMs.

This demonstrated an important grounding behavior:

When the supplied context does not support an answer, the model can refuse to invent information outside the retrieved knowledge.

A second question asked about how AI models work.

This time, the retrieved context contained relevant stored information about:

- Matrix operations
- Model weights
- Backpropagation
- Gradient descent
- Embedding models
- RAG systems

The LLM successfully used the retrieved context to produce an answer.

This confirmed that all major components were functioning together:

Question → Retrieval → Context Extraction → Prompt Construction → Local LLM → Answer

---

Retrieval Limitation Discovered

End-to-end testing also exposed an important limitation of the current retrieval implementation.

ChromaDB always returns the nearest matching vectors when results are requested.

However:

Nearest Match ≠ Sufficiently Relevant Match

For example, a question about LLMs may retrieve general AI-model notes because they are semantically related even if they do not contain enough information to answer the exact question.

The current system does not yet apply a minimum relevance threshold.

Future retrieval improvements may use ChromaDB distance values to determine whether a retrieved chunk is sufficiently relevant before including it in the prompt.

This will help distinguish between:

Relevant Knowledge

and:

Best Available Match That Is Still Weak

This will be especially important for the project's strict personal-knowledge mode.

Current RAG Architecture

The completed Sprint 2 architecture is:

Knowledge Ingestion

Raw Note

→ ingest_note()

→ chunk_text()

→ SentenceSplitter

→ Individual Chunks

→ EmbeddingGemma through Official Ollama:11435

→ Embedding Vectors

→ ChromaDB

Question Retrieval

User Question

→ EmbeddingGemma through Official Ollama:11435

→ Query Embedding

→ ChromaDB Similarity Search

→ Relevant Stored Chunks

Answer Generation

Retrieved Chunks + User Question

→ Prompt Builder

→ Qwen3 4B through IPEX-LLM:11434

→ Intel Arc GPU

→ Generated Answer

Agent Orchestration

ask(question)

coordinates retrieval, prompt construction, and response generation.

Sprint 2 Result

Sprint 2 successfully achieved its original goal:

Build the first working Retrieval-Augmented Generation pipeline using local models and locally stored knowledge.

The originally targeted workflow was:

User Question → Embedding Model → Vector Database Search → Relevant Note Chunks → Prompt Construction → Local LLM → Grounded Response

That complete workflow is now operational.

The system can:

- Accept long notes.
- Divide notes into searchable chunks.

- Generate embeddings locally.
- Store embeddings and text persistently in ChromaDB.
- Generate query embeddings.
- Perform semantic similarity retrieval.
- Retrieve the original text associated with matching vectors.
- Construct an LLM prompt using retrieved knowledge.
- Send the prompt to a locally running Qwen model.
- Generate an answer using the retrieved context.
- Run EmbeddingGemma and Qwen through separate local Ollama-compatible runtimes.
- Use Intel Arc GPU acceleration for Qwen generation through IPEX-LLM.

The first functional end-to-end RAG pipeline has therefore been completed.

Sprint 2 Status

Sprint: Sprint 2

Goal: First Working RAG Pipeline

Status: COMPLETE

The following improvements remain available for future sprints rather than being required for completion of the first RAG implementation:

- Retrieval relevance thresholds
- Improved grounding instructions
- Better handling of weak retrieval results
- Cleaner suppression of Qwen <think> output
- Output-length control
- More advanced prompt design
- Conversation-memory integration
- Retrieval of conversation history
- Neighboring-chunk retrieval
- Hybrid retrieval

- Reranking
- Backend API integration

These improvements will build on the working RAG foundation completed during Sprint 2.